

Les listes

Une liste est un objet permettant de stocker et de structurer plusieurs objets de mêmes ou de différents types; des nombres, des chaînes de caractères, des booléens, ... et d'autres listes.

Les éléments d'une liste sont mis entre crochets séparés par des virgules.

1 Création d'une liste

Une liste vide :

```
[1]: ma_liste = []
```

Des listes d'éléments de même type :

```
[2]: nombres = [0, 1, 2, 3, 4, 5, 6, 7]
courses = ['farine de blé', 'lait', 'thé', 'pâtes', 'fruits']
```

Des listes d'éléments de types variés :

```
[3]: import math

ma_liste = ['cercle', 7, math.pi, True]
```

2 Accès aux éléments d'une liste

Par index positif (les index commencent à 0) :

```
[4]: print("Le premier élément de la liste :", courses[0])
print("Le deuxième élément de la liste :", courses[1])
```

Le premier élément de la liste : farine de blé

Le deuxième élément de la liste : lait

Par index négatif pour accéder aux éléments à partir de la fin :

```
[5]: print("Le dernier élément de la liste :", courses[-1])
print("L'avant dernier :", courses[-2])
```

Le dernier élément de la liste : fruits

L'avant dernier : pâtes

3 Opérations sur les listes

Soit les deux listes :

```
[6]: liste1 = [1, 2, 3]
     liste2 = [4, 5, 6]
```

Concaténer les listes en utilisant l'opérateur + :

```
[7]: liste = liste1 + liste2
     liste
```

```
[7]: [1, 2, 3, 4, 5, 6]
```

Répéter une liste en utilisant l'opérateur * :

```
[8]: 3*liste
```

```
[8]: [1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6]
```

Tester l'appartenance avec l'opérateur *in* :

```
[9]: 'fruits' in courses
```

```
[9]: True
```

Modifier un élément à partir de son indice :

```
[10]: courses[3]='salade'
      courses
```

```
[10]: ['farine de blé', 'lait', 'thé', 'salade', 'fruits']
```

4 Quelques fonctions

- *len(liste)* : Renvoie la longueur (nombre d'éléments) de la liste.
- *max(liste)* : Renvoie le plus grand élément de la liste
- *min(liste)* : Renvoie le plus petit élément de la liste.

```
[11]: len(courses)
```

```
[11]: 5
```

```
[12]: max(courses) #selon l'ordre alphabétique
```

```
[12]: 'thé'
```

```
[13]: min(nombres)
```

```
[13]: 0
```

5 Parcours d'une liste

par valeur :

```
[14]: for x in courses:
      print(x)
```

```
farine de blé
lait
thé
salade
fruits
```

par indice :

```
[15]: for i in range(len(courses)):
      print(courses[i])
```

```
farine de blé
lait
thé
salade
fruits
```

par les deux en même temps :

```
[16]: for i, x in enumerate(courses):
      print(i, ":", x)
```

```
0 : farine de blé
1 : lait
2 : thé
3 : salade
4 : fruits
```

6 Quelques méthodes

- *liste.append(x)* : Ajouter **x** à la fin de la liste.
- *liste1.extend(liste2)* : Ajouter les éléments de la **liste2** à la fin de la **liste1**.
- *liste.insert(i, x)* : Insérer un élément **x** à la position **i**.
- *liste.remove(x)* : Supprimer de la liste le premier élément dont la valeur est égale à **x**.
- *liste.index(x)* : Renvoyer la position du premier élément de la liste dont la valeur égale **x** (en commençant à compter les positions à partir de zéro).
- *liste.pop(i)* : Retirer et renvoyer l'élément d'indice **i**; et *liste.pop()* pour retirer et renvoyer le dernier élément.
- *liste.count(x)* : Renvoyer le nombre d'éléments ayant la valeur **x**.
- *liste.reverse()* : Inverser l'ordre des éléments.
- *liste.sort()* : Trier les éléments.

```
[17]: courses = ['farine de blé', 'lait', 'thé', 'pâtes', 'salade', 'fruits']  
      # Ajouter un élément à la fin de la liste  
      courses.append('pâtes')  
      courses
```

```
[17]: ['farine de blé', 'lait', 'thé', 'pâtes', 'salade', 'fruits', 'pâtes']
```

```
[18]: # Ajouter les éléments d'une autre liste à la fin de la liste  
      courses.extend(['chocolat', 'biscuits'])  
      courses
```

```
[18]: ['farine de blé',  
      'lait',  
      'thé',  
      'pâtes',  
      'salade',  
      'fruits',  
      'pâtes',  
      'chocolat',  
      'biscuits']
```

```
[19]: # Insérer à un index spécifique  
      courses.insert(2, 'dattes')  
      courses
```

```
[19]: ['farine de blé',  
      'lait',  
      'dattes',  
      'thé',  
      'pâtes',  
      'salade',  
      'fruits',  
      'pâtes',  
      'chocolat',  
      'biscuits']
```

```
[20]: # Supprimer la première occurrence d'un élément par sa valeur  
      courses.remove('pâtes')  
      courses
```

```
[20]: ['farine de blé',  
      'lait',  
      'dattes',  
      'thé',  
      'salade',  
      'fruits',  
      'pâtes',  
      'chocolat',
```

```
'biscuits']
```

```
[21]: # Trouver l'index de la première occurrence d'une valeur dans une liste
i = courses.index('lait')

# Retirer et obtenir un élément de la liste par son index
x = courses.pop(i)
print(x)
```

```
lait
```

```
[22]: # Inverser l'ordre des éléments
courses.reverse()
courses
```

```
[22]: ['biscuits',
       'chocolat',
       'pâtes',
       'fruits',
       'salade',
       'thé',
       'dattes',
       'farine de blé']
```

```
[23]: # Trier la liste par ordre alphabétique (croissant)
courses.sort()
courses
```

```
[23]: ['biscuits',
       'chocolat',
       'dattes',
       'farine de blé',
       'fruits',
       'pâtes',
       'salade',
       'thé']
```

```
[24]: nombres = [0, 1, 2, 3, 5, 8, 13, 21]

# Trier la liste par ordre décroissant
nombres.sort(reverse=True)
nombres
```

```
[24]: [21, 13, 8, 5, 3, 2, 1, 0]
```

7 Tranchage (Slicing)

Pour obtenir une tranche d'indice entre *début* (inclus) et *fin* (exclus) avec un *pas*, on utilise `liste[début : fin : pas]`. La liste obtenue est composée de `liste[début]`, `liste[début + 1 × pas]`, `liste[début + 2 × pas]`, ... et `liste[début + nombre × pas]`, tel que `début + nombre × pas` tend vers *fin* sinon on obtient une liste vide `[]`.

```
[25]: liste = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

      liste[1:7:1]
```

```
[25]: [1, 2, 3, 4, 5, 6]
```

Les valeurs par défaut pour le début, la fin et le pas sont respectivement 0, la longueur de la liste et 1 :

```
[26]: liste[::]
```

```
[26]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[27]: liste[::2]
```

```
[27]: [0, 2, 4, 6, 8]
```

Dans le cas d'un pas négatif les valeurs par défaut pour le début et la fin sont respectivement (-1) et (- longueur - 1) :

```
[28]: liste[::-2]
```

```
[28]: [9, 7, 5, 3, 1]
```

```
[29]: liste[0:6:-1]
```

```
[29]: []
```

8 Listes en compréhension

Les listes en compréhension fournissent un moyen de construire des listes de manière concise et élégante. Elles permettent d'écrire des boucles, des conditions, et des opérations sur les éléments d'une liste en une seule ligne de code :

liste1 = [*expression* for *élément* in *liste2* if *condition*]

L'évaluation de l'expression sera ajoutée à la ***liste1*** pour chaque élément de la ***liste2*** avec une condition à vérifier.

```
[30]: # Ajouter 1 à chaque élément d'une autre liste

      ancienne_liste = [1, 2, 3, 4]
      nouvelle_liste = [x+1 for x in ancienne_liste]
```

```
nouvelle_liste
```

```
[30]: [2, 3, 4, 5]
```

```
[31]: # Une liste des carrés des nombres de 0 à 10
```

```
squares = []  
for x in range(11):  
    squares.append(x*x)  
  
squares
```

```
[31]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
[32]: #ou, de manière équivalente :
```

```
squares = [x*x for x in range(11)]  
  
squares
```

```
[32]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Ce qui est plus court et plus lisible.

Ajout de conditions :

```
[33]: # les nombres pairs seulement
```

```
even_squares = [x*x for x in range(11) if x%2 == 0]  
  
even_squares
```

```
[33]: [0, 4, 16, 36, 64, 100]
```

9 TP

Exercice 1

Soit `liste = [0,1,2,3,5,8,13,21,34,55,89]`

Donner les résultats des instructions suivantes :

- `liste[1:7]`
- `liste[:]`
- `liste[3:1]`
- `liste[-3:-1]`
- `liste[:-2]`
- `liste[6:]`
- `liste[:3]`
- `liste[::3]`

Exercice 2

Créer une fonction `cube(liste)` qui renvoie une liste contenant le cube de chaque élément de la liste originelle.

Exercice 3

- Créer une fonction `somme(prix)` qui renvoie le prix total à payer à partir d'une liste de prix.
- Créer une fonction `prix_article(noms, prix, nom_article)` qui renvoie le prix d'un article donné à partir d'une liste de noms d'articles et la liste des prix correspondante.

Exemple :

En utilisant les deux listes :

`noms = ['farine de blé 10kg', 'lait 1/2l', 'thé 200g', 'pâtes 500g']; prix = [50, 3.5, 25, 20]`,

l'appel `prix_article(noms, prix, 'lait 1/2l')` renvoie le prix `3.5`.

Exercice 4

Créer une fonction `nombre_occurences(valeur, liste)` qui compte les occurrences d'une valeur dans une liste.

Exercice 5

1. Créer une fonction `recherche_squentielle(liste, valeur)` qui renvoie l'indice de la première occurrence de la valeur dans la liste si elle existe, et -1 sinon.
2. Créer une fonction `recherche_dichotomique(liste, valeur)` qui prend une liste triée et une valeur et renvoie `True` si la valeur se trouve dans la liste, et `False` sinon.

Principe :

On divise la liste en deux sous-listes et on compare l'élément du milieu de la liste avec la valeur recherchée :

- Si c'est la bonne valeur, on renvoie `True`.
- Si la valeur est plus petite, on recherche dans la sous-liste gauche.

- Si la valeur est plus grande, on recherche dans la sous-liste droite.

On recommence jusqu'à trouver l'élément ou jusqu'à ce que la sous-liste soit vide et on renvoie *False*.

Exercice 6

Créer une fonction *maximum(liste)* qui renvoie le maximum d'une liste de nombres.

10 Résolution

```
[34]: # Exercice 1
```

```
liste = [0,1,2,3,5,8,13,21,34,55,89]
```

```
[35]: liste[1:7]
```

```
[35]: [1, 2, 3, 5, 8, 13]
```

```
[36]: liste[:]
```

```
[36]: [0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

```
[37]: liste[3:1]
```

```
[37]: []
```

```
[38]: liste[-3:-1]
```

```
[38]: [34, 55]
```

```
[39]: liste[:-2]
```

```
[39]: [0, 1, 2, 3, 5, 8, 13, 21, 34]
```

```
[40]: liste[6:]
```

```
[40]: [13, 21, 34, 55, 89]
```

```
[41]: liste[::3]
```

```
[41]: [0, 3, 13, 55]
```

```
[42]: liste[::-3]
```

```
[42]: [89, 21, 5, 1]
```

```
[43]: # Exercice 2
```

```
def cube1(liste):  
    cubes = []  
    for x in liste:  
        cubes.append(x**3)    #ajout du cube du nombre courant  
    return cubes
```

```
#ou bien
```

```
def cube2(liste):
```

```

    return [x**3 for x in liste]

# Exemple

nombres = [0, 1, 1, 2, 3, 5, 8, 13]

cube2(nombres)

```

[43]: [0, 1, 1, 8, 27, 125, 512, 2197]

```

[44]: # Exercice 3.1

def somme(prix):
    pt = 0    #initialisation de l'accumulateur
    for p in prix:
        pt += p    #ajout du prix courant aux prix cumulés
    return pt

# Exemple

prix = [50, 99.99, 10, 27, 250, 3.7]

somme(prix)

```

[44]: 440.69

```

[45]: # Exercice 3.2

def prix_article(noms, prix, nom_article):
    i = noms.index(nom_article)    #extraction de l'indice
    return prix[i]                #renvoie du prix situé à l'indice extrait

# Exemple

noms = ['farine de blé 10kg', 'lait 1/2l', 'thé 200g', 'dattes 1kg']
prix = [100, 3.5, 27, 50]

prix_article(noms, prix, 'thé 200g')

```

[45]: 27

```

[46]: # Exercice 4

def nombre_occurrences(valeur, liste):
    c = 0    #initialisation du compteur
    for x in liste:
        if x == valeur:

```

```

        c += 1    #ajout de 1 au compteur si la valeur est trouvée
    return c

nombres = [0, 1, 1, 2, 3, 5, 8, 13, 1]

nombre_occurrences(1, nombres)

```

[46]: 3

```

[47]: # Exercice 5.1

def recherche_séquentielle(liste, valeur):
    for i in range(len(liste)):
        if liste[i] == valeur:
            return i
    return -1

```

```

[48]: # Exercice 5.2

def recherche_dichotomique(liste, valeur):
    début = 0
    fin = len(liste)-1
    while début<=fin:
        milieu=(début+fin)//2
        if valeur>liste[milieu]:
            début=milieu+1
        elif valeur<liste[milieu]:
            fin=milieu-1
        else:
            return True
    return False

```

```

[49]: # Exemple

ma_liste = [i for i in range(1000001)]

recherche_séquentielle(ma_liste, 1000000)
#nécessite 1 000 000 itérations au pire des cas (l'élément recherché se trouve
↪ à la fin de la liste)

```

[49]: 1000000

```

[50]: recherche_dichotomique(ma_liste, 1000000)

#nécessite seulement (log à base 2 de 1 000 000) itérations, soit 20 itérations.

```

[50]: True

```
[51]: # Exercice 6

def maximum(liste):
    maxi = liste[0]    #initialisation du maximum par le premier élément de la
    ↪ liste
    for i in range(1, len(liste)):
        if liste[i] > maxi:
            maxi = liste[i]
    return maxi

# Exemple

prix = [50, 99.99, 10, 27, 250, 3.7]

maximum(prix)
```

[51]: 250